

Version Control Management System

Assignment Specification Document

1. Overview

This assignment requires building a full-stack Version Control Management application that lets teams track versioned releases (major / minor / bug-fix) across multiple projects. The system covers authentication, listing with search/filter/sort/pagination, CRUD with soft-delete, and automatic version-number computation.

2. Tech Stack

Layer	Technology
Frontend	ReactJS + TypeScript
Backend	Node.js + TypeScript
Database	MySQL
ORM	Sequelize (TypeScript models, as per provided schema)
Auth	JWT (Access Token + Refresh Token)
Backend run mode	Compile TS → run compiled dist/server.js (never run server.ts directly)

3. Authentication & Authorization

- Login page with Email Address and Password fields.
- On successful login, issue:
 - Access Token — validity 1 hour
 - Refresh Token — validity 6 hours
- Refresh token flow must silently reissue a new access token when it expires, without forcing re-login, until the refresh token itself expires.
- All protected APIs must validate the JWT access token; expired/invalid tokens return 401.
- Default landing page after login: Version Listing page.
- Logout button placed at the bottom of the sidebar; clears tokens and redirects to Login.

4. Application Layout

4.1 Sidebar

- Collapsible sidebar (expand/collapse toggle).
- Menu option: “Version Listing” — navigates to the listing page (also the default post-login route).
- Logout button fixed at the bottom of the sidebar.

5. Version Listing Page

5.1 Default Behaviour

- Lists all version_control entries sorted by created_at in descending order.

- When the Project filter dropdown has no value selected, return entries across all projects and all versions, mixed together, ordered by created_at desc.

5.2 Show Deleted Toggle

- A “show deleted” control that, when enabled, includes soft-deleted (is_active = false) records in the listing.
- Implemented as an optional query parameter (e.g. showDeleted=true/false) sent to the backend; when omitted/false, only active records are returned.

5.3 Search

- A search box searches across the listing (e.g. version title / version info).
- If no Project dropdown value is selected — search across all projects.
- If a Project dropdown value is selected — search only within that project’s records.

5.4 Filters (Dropdowns)

- Project dropdown — filters listing/search by selected project.
- Date range filter — Start Date and End Date pickers; returns records with created_at between the selected range.
- Version dropdown — filters by version type enum: major, minor, bug-fix.

5.5 Pagination

- Server-side pagination: backend returns a page of records plus total count.
- Client-side controls: page number navigation, default page size of 10 records.

5.6 Row Actions

- View — opens the record in read-only modal.
- Edit — opens the Add/Edit modal pre-filled with the record’s data.
- Soft Delete — a toggle button per row that flips is_active (soft delete / restore) instead of physically deleting the row.

6. Add / Edit Modal Form

6.1 Fields

Field	Type	Editable?	Notes
Project	Dropdown	Yes	Sourced from project_master (active projects)
Version	Dropdown (enum)	Yes	major / minor / bug-fix
Version Number	Text (read-only)	No	Auto-computed — see rule below
Title	Textbox	Yes	Maps to version_title
Version Info	Textarea	Yes	Maps to version_info (stored as JSON)

6.2 Version Number Auto-Increment Rule

When the user selects a Project and a Version type, the Version Number field is automatically computed (and remains non-editable in both Add and Edit forms):

1. Query the database for the most recent version_control entry for the selected project (highest version_number / latest created_at).
2. If no previous entry exists for that project, initialize as 1.0.0.
3. If a previous entry exists, increment the segment corresponding to the selected version type, following semantic-versioning style (MAJOR.MINOR.BUGFIX):

major → increment the 1st segment, reset 2nd and 3rd segments to 0 (e.g. 1.4.2 → 2.0.0)

minor → increment the 2nd segment, reset 3rd segment to 0 (e.g. 1.4.2 → 1.5.0)

bug-fix → increment the 3rd segment only (e.g. 1.4.2 → 1.4.3)

4. Set the computed value into the read-only Version Number field before save.

This computation should be re-triggered whenever the Project or Version dropdown changes, and re-validated on the backend before persisting (do not trust the client-computed value alone).

7. Database Schema

7.1 role_master

Column	Type	Notes
id	INT (PK)	Auto increment
role_name	VARCHAR	—
description	VARCHAR / TEXT (nullable)	—
is_active	BOOLEAN	—
created_on	DATETIME	—

7.2 user_master

Column	Type	Notes
id	INT (PK)	—
role_id	INT (FK → role_master)	—
first_name	VARCHAR	—
last_name	VARCHAR	—
email_address	VARCHAR	Used for login
password	VARCHAR	Hashed
password_reset_token	VARCHAR (nullable)	—
password_reset_expiry	DATETIME (nullable)	—
is_active	BOOLEAN	—
created_by	INT	—
created_on	DATETIME	—
updated_by	INT (nullable)	—
updated_on	DATETIME (nullable)	—

7.3 project_master

Column	Type	Notes
id	INT (PK)	—
project_name	VARCHAR	—
is_active	BOOLEAN	—
created_by	INT	—
created_at	DATETIME	—
updated_by	INT (nullable)	—
updated_at	DATETIME (nullable)	—

7.4 version_control

Column	Type	Notes
id	INT (PK)	—
project_id	INT (FK → project_master)	—
version	ENUM	major / minor / bug-fix
version_number	VARCHAR(10)	Auto-computed, e.g. 1.0.1
version_title	VARCHAR(100)	—
version_info	JSON	—
is_active	BOOLEAN	Soft-delete flag
created_by	INT (FK → user_master)	—
created_at	DATETIME	—
updated_by	INT (nullable)	—
updated_at	DATETIME (nullable)	—

8. API Requirements (Suggested)

Method	Endpoint	Description
POST	/api/auth/login	Login with email & password; returns access + refresh tokens
POST	/api/auth/refresh-token	Issue new access token using refresh token
POST	/api/auth/logout	Invalidate session/tokens
GET	/api/versions	List with query params: page, limit, projectId, versionType, startDate, endDate, search, showDeleted
GET	/api/versions/:id	Get single record (view)
POST	/api/versions	Create new version entry (server computes version_number)
PUT	/api/versions/:id	Edit existing entry
PATCH	/api/versions/:id/toggle-status	Soft delete / restore (toggles is_active)
GET	/api/projects	List active projects (for dropdown)
GET	/api/versions/next-number	Given projectId + versionType, return computed next version_number

9. Functional Requirements Summary

- Listing sorted by created_at DESC by default.
- showDeleted optional query param toggles inclusion of soft-deleted rows.
- No project selected → all projects/versions mixed together.
- Search scoped to selected project when one is chosen, else global.
- Frontend + backend pagination, default page size 10.
- Project, Version-type, and Date-range dropdown filters.
- Version Number field is always read-only and system-computed.
- Soft delete implemented as a status toggle, not a hard delete.
- JWT-based auth with 1h access / 6h refresh token lifetimes.
- Collapsible sidebar with Version Listing link and Logout at the bottom.

10. Deliverables

5. Complete source code (frontend + backend) in TypeScript.

6. Backend must be built and run from the compiled dist/server.js (not ts-node/server.ts).
7. Database migration/seed scripts for all four tables.
8. README with setup steps, environment variables, and build/run instructions.
9. Short write-up or Postman collection covering the API endpoints.

11. Evaluation Criteria

- Correctness of listing filters, search-scoping, and sort order.
- Correctness of the version-number auto-increment logic.
- Proper soft-delete handling (data retained, filtered by is_active).
- JWT auth implementation and token refresh handling.
- Code organization, TypeScript typing quality, and UI usability (collapsible sidebar, modal forms, pagination).